

Fixed-Branch Analytical Derivative for the Zhao–Cui Tensor-Train Sequential Filter

BayesFilter P11 Derivation Note

2026-05-30

Abstract

This note derives the analytical derivative of the scalar produced by a fixed-branch Zhao–Cui squared tensor-train sequential filter. The derivative is for the declared approximate scalar

$$\hat{\ell}_T^{\text{TT}}(\alpha) = \sum_{t=1}^T \{\log \hat{z}_t(\alpha) - c_t(\alpha)\},$$

where α is an external model parameter, $\hat{z}_t$ is the squared-TT/SIRT normalizer, and c_t is the stabilizing constant used in the companion code’s `log(sirt.z)-const` accumulator. The derivation is complete only for a frozen algorithmic branch: fixed variable ordering, fixed bases, fixed domains, fixed interpolation or cross sets, fixed ranks, fixed rounding decisions, fixed random/debug samples, fixed defensive term policy, fixed preconditioner choice, and fixed selected minimizer for c_t . It does not claim a complete derivative for the adaptive stochastic companion code.

1 Anchors And Scope

The source construction is Zhao–Cui’s sequential TT method. Their Algorithm 1 forms an unnormalized density

$$q_t(x_t, \vartheta, x_{t-1}) = \hat{\pi}_{t-1}(x_{t-1}, \vartheta \mid y_{1:t-1}) f(x_t \mid x_{t-1}, \vartheta) g(y_t \mid x_t, \vartheta), \quad (\text{ZC-12})$$

then reapproximates it by a TT and integrates the TT to obtain a normalizing constant. Their squared-TT construction approximates the square root of an unnormalized density by

$$\sqrt{\pi(x)} \approx \phi(x) = H_1(x_1) \cdots H_m(x_m),$$

and defines the nonnegative defensive approximation

$$\hat{\pi}(x) = \phi(x)^2 + \tau \lambda(x), \quad \hat{z} = \int \hat{\pi}(x) \, dx. \quad (\text{ZC-13})$$

Algorithm 2 applies this construction to the sequential density q_t , using the square-root TT ϕ_t and defensive density

$$\hat{\pi}_t(x_t, \vartheta, x_{t-1}) = \phi_t(x_t, \vartheta, x_{t-1})^2 + \tau_t \lambda(x_t) \lambda(\vartheta) \lambda(x_{t-1}). \quad (\text{ZC-16})$$

Zhao–Cui Section 4.1 identifies the exact joint normalizer as evidence and explains that the algorithm approximates q_t because the exact marginal posterior entering the recursion is not directly evaluable.

The inspected companion code implements the same scalar path through:

```
models/full_sol.m                sol.logmarginal_likelihood += log(sirt.z)-const,
deep-tensor.dev/src/SIRT.m       potential_to_density,
deep-tensor.dev/src/@TTSIRT/TTSIRT.m approx = TTFun(func,...),
deep-tensor.dev/src/@TTSIRT/marginalise.m obj.z = obj.fun_z + obj.tau.
```

The P10 Octave audit snapshot under `third_party/audit/zhao_cui_tensor_ssm_p10/source` shows that a reduced Kalman path reaches the TT/SIRT approximation, completes one ALS pass, and reports a finite log-marginal-likelihood accumulator. That smoke is implementation evidence, not a proof of posterior accuracy.

Parameter notation. Zhao–Cui use θ for the learned state-space parameter inside the sequential posterior. This note writes that variable as ϑ . The external parameter with respect to which we differentiate the approximate scalar is α_i , and $\dot{h}$ means $\partial_i h$. If a future BayesFilter implementation samples ϑ directly by HMC, it must first translate this external-parameter derivative into the chosen target parameterization.

2 Declared Scalar

Let

$$u_t = (x_t, \vartheta, x_{t-1}) \in \mathbb{R}^D.$$

At time t , the unnormalized density supplied to the squared-TT step is

$$q_t(u_t; \alpha) = \hat{p}_{t-1}^{\text{TT}}(x_{t-1}, \vartheta; \alpha) f_\alpha(x_t \mid x_{t-1}, \vartheta) g_\alpha(y_t \mid x_t, \vartheta).$$

The previous approximation $\hat{p}_{t-1}^{\text{TT}}$ is the marginal of the previous squared-TT density after normalization. At $t = 1$, it is replaced by the prior and initial-state density used by the model.

The code constructs an affine approximation coordinate

$$u_t = A_t r_t + b_t,$$

with $A_t = \text{L_temp}$ and $b_t = \text{mu_temp}$. In the companion code, A_t and b_t are estimated from weighted samples and are therefore branch-changing unless frozen. In this note they are fixed smooth branch objects. The unshifted potential in r -coordinates is

$$V_{t,0}(r; \alpha) = -\log q_t(A_t r + b_t; \alpha) - \log |\det A_t|.$$

The code then chooses a stabilizing constant

$$c_t(\alpha) = V_{t,0}(r_{t,s^*}; \alpha)$$

when the selected minimizer s^* of the fixed random/debug grid is frozen and unique. If the implementation freezes the numerical value of c_t instead, then $\dot{c}_t = 0$, but the differentiated scalar is a different declared scalar. The shifted potential is

$$V_t(r; \alpha) = V_{t,0}(r; \alpha) - c_t(\alpha).$$

The SIRT implementation approximates a square-root density in the reference coordinates used by the basis. Let $\rho_t(r; \alpha)$ denote the unnormalized reference-coordinate density after all fixed basis-domain Jacobian and reference-measure factors used by `SIRT.potential_to_density` have been included. Then

$$\psi_t(r; \alpha) = \rho_t(r; \alpha)^{1/2}$$

is the function approximated by `TTFun`. In the simplest affine and fixed-basis case,

$$\dot{\psi}_t(r; \alpha) = -\frac{1}{2} \psi_t(r; \alpha) \dot{V}_t(r; \alpha),$$

with additional Jacobian or reference-density derivative terms only if the basis-domain map itself depends on α .

The fixed-branch squared-TT approximation is

$$\phi_t(r; \alpha) = G_{t,1}(r_1; \alpha) G_{t,2}(r_2; \alpha) \cdots G_{t,D}(r_D; \alpha),$$

where $G_{t,k}(r_k; \alpha) \in \mathbb{R}^{r_{k-1} \times r_k}$, $r_0 = r_D = 1$, all ranks are fixed, and every basis/interpolation decision is fixed. The defensive squared-TT normalizer is

$$\hat{z}_t(\alpha) = \int \phi_t(r; \alpha)^2 d\nu(r) + \tau_t(\alpha),$$

where ν is the fixed tensor-product basis/reference measure and τ_t is the defensive mass term. Thus the declared scalar is

$$\hat{\ell}_T^{\text{TT}}(\alpha) = \sum_{t=1}^T \{\log \hat{z}_t(\alpha) - c_t(\alpha)\}.$$

Regularity assumptions. The formulas below require the usual finite-dimensional sensitivity conditions on the frozen branch. The transition and observation densities, the previous TT marginal, and the fixed coordinate maps are differentiable in α on the evaluated domain; the normalizers $R_{t,0} + \tau_t$ are finite and positive; the support does not change pathologically with α ; the fixed local linear or least-squares systems are nonsingular or have a declared differentiable pseudoinverse branch; and differentiation may be interchanged with the finite contractions or with the stated integrals. If QR, SVD, Cholesky, or inverse-CDF solves are differentiated, their active rank, sign, bracket, and simple-spectrum branches are part of the fixed branch.

3 Derivative Of The Input Density

From (2),

$$\log \dot{q}_t = \log \dot{\hat{p}}_{t-1}^{\text{TT}} + \log \dot{f}_\alpha(x_t \mid x_{t-1}, \vartheta) + \log \dot{g}_\alpha(y_t \mid x_t, \vartheta).$$

The model-score terms are ordinary transition and likelihood derivatives. The recursive term is more delicate. If

$$\hat{p}_{t-1}^{\text{TT}}(v; \alpha) = \frac{a_{t-1}(v; \alpha)}{\hat{z}_{t-1}(\alpha)}$$

is the marginal density over $v = (x_{t-1}, \vartheta)$, then

$$\log \dot{\hat{p}}_{t-1}^{\text{TT}}(v; \alpha) = \frac{\dot{a}_{t-1}(v; \alpha)}{a_{t-1}(v; \alpha)} - \frac{\dot{\hat{z}}_{t-1}(\alpha)}{\hat{z}_{t-1}(\alpha)}.$$

The marginal numerator a_{t-1} is computed by contracting the previous TT cores over the eliminated coordinates. Its derivative is obtained by the same contraction with one differentiated core at a time, plus any derivative of $\tau_{t-1}\lambda$ if the defensive term or reference density depends on α .

If A_t , b_t , or the selected constant point r_{t,s^*} depend smoothly on α , then

$$\begin{aligned}\dot{V}_{t,0}(r) &= -\log q_t(u_t) - \nabla_u \log q_t(u_t)^\top (\dot{A}_t r + \dot{b}_t) - \text{tr}(A_t^{-1} \dot{A}_t), \\ \dot{c}_t &= \dot{V}_{t,0}(r_{t,s^*}) + \nabla_r V_{t,0}(r_{t,s^*})^\top \dot{r}_{t,s^*}.\end{aligned}$$

For the companion code's random grid, r_{t,s^*} is fixed after branch freezing, so $\dot{r}_{t,s^*} = 0$. If the minimizing index changes, the ordinary derivative is not the derivative of a single smooth branch.

4 Fixed-Branch TT Core Sensitivities

Equation (2) gives the product-rule derivative

$$\dot{\phi}_t(r) = \sum_{k=1}^D G_{t,1}(r_1) \cdots \dot{G}_{t,k}(r_k) \cdots G_{t,D}(r_D).$$

The missing implementation object is therefore $\dot{G}_{t,k}$ for each fixed core. A fixed TT-cross or ALS branch can be written as a finite system of interpolation or least-squares equations. Let $g_{t,k}$ be the vector of free coefficients in core k , with all ranks, basis functions, sample points, sweep order, and neighboring interfaces fixed.

For an exact fixed interpolation equation,

$$A_{t,k}(\alpha) g_{t,k}(\alpha) = b_{t,k}(\alpha),$$

the sensitivity is

$$A_{t,k} \dot{g}_{t,k} = \dot{b}_{t,k} - \dot{A}_{t,k} g_{t,k}.$$

Here $b_{t,k}$ consists of target square-root evaluations $\psi_t(r^{(j)}; \alpha)$ after subtracting or conditioning on the fixed left/right interfaces, and $\dot{b}_{t,k}$ uses (2). The matrix $A_{t,k}$ contains basis evaluations and interface products; $\dot{A}_{t,k}$ is zero only when the interfaces are held numerically fixed. If interfaces from earlier core updates are differentiated, $\dot{A}_{t,k}$ contains their sensitivities.

For a weighted least-squares core update,

$$g_{t,k} = \arg \min_g \|W_{t,k}^{1/2} (A_{t,k} g - b_{t,k})\|_2^2,$$

with fixed weights and full-rank normal matrix $N_{t,k} = A_{t,k}^\top W_{t,k} A_{t,k}$, the normal-equation derivative is

$$N_{t,k} \dot{g}_{t,k} = \dot{A}_{t,k}^\top W_{t,k} (b_{t,k} - A_{t,k} g_{t,k}) + A_{t,k}^\top W_{t,k} (\dot{b}_{t,k} - \dot{A}_{t,k} g_{t,k}).$$

If the residual vanishes, this reduces to the interpolation formula. If rank truncation, pivot selection, enrichment samples, or ALS stopping change, then (4)–(4) describe only the derivative of the frozen local branch.

5 Normalizer Contraction And Its Derivative

Let $\{b_{k,a}\}_{a=1}^{n_k}$ be the fixed one-dimensional basis on coordinate k , and let

$$M_k[a, b] = \int b_{k,a}(r_k) b_{k,b}(r_k) d\nu_k(r_k)$$

be the one-dimensional mass matrix. Write the coefficient core slices as

$$C_{t,k,a}(\alpha) \in \mathbb{R}^{r_{k-1} \times r_k}.$$

Then

$$G_{t,k}(r_k; \alpha) = \sum_{a=1}^{n_k} C_{t,k,a}(\alpha) b_{k,a}(r_k).$$

Define the right contractions

$$R_{t,D} = 1, \quad R_{t,k-1} = \sum_{a,b=1}^{n_k} M_k[a, b] C_{t,k,a} R_{t,k} C_{t,k,b}^\top, \quad k = D, \dots, 1.$$

The square part of the normalizer is the scalar

$$z_{t,\phi} = R_{t,0}.$$

This is the direct mass-matrix form of the normalizer. The companion code's `@TTSIRT/marginalise.m` computes the same quantity in exact arithmetic through repeated mass-weighted QR contractions and then sets

$$\texttt{sirt.z} = \texttt{sirt.fun.z} + \texttt{sirt.tau}.$$

For a strict same-code-scalar derivative, one may differentiate the QR branch. For a cleaner implementation, compute both value and derivative by the direct contraction (5); the scalar must then be declared as that contraction rather than as a black-box QR floating-point value.

Because the basis is fixed in this note, $\dot{M}_k = 0$. The contraction derivative is

$$\begin{aligned} \dot{R}_{t,k-1} = \sum_{a,b=1}^{n_k} M_k[a, b] & \left[\dot{C}_{t,k,a} R_{t,k} C_{t,k,b}^\top + C_{t,k,a} \dot{R}_{t,k} C_{t,k,b}^\top \right. \\ & \left. + C_{t,k,a} R_{t,k} \dot{C}_{t,k,b}^\top \right], \quad k = D, \dots, 1. \end{aligned}$$

Thus

$$\dot{\hat{z}}_t = \dot{R}_{t,0} + \dot{\tau}_t.$$

Equivalently, in integral notation,

$$\dot{\hat{z}}_t = 2 \int \phi_t(r) \dot{\phi}_t(r) d\nu(r) + \dot{\tau}_t.$$

6 Final Score

Combining (2) and (5), the fixed-branch analytical score is

$$\dot{\hat{\ell}}_T^{\text{TT}} = \sum_{t=1}^T \left[\frac{\dot{R}_{t,0} + \dot{\tau}_t}{R_{t,0} + \tau_t} - \dot{c}_t \right].$$

Every term in (6) is the derivative of the declared scalar only under the fixed-branch assumptions. If an implementation uses the companion code’s branch-changing choices, then the derivative is not an ordinary HMC gradient unless those choices are frozen, smoothed, marginalized, or moved outside the sampler.

7 Implementation Checklist

A future same-scalar prototype must implement the following objects.

Object	Required derivative or declaration
Model densities	$\log \dot{f}_\alpha$, $\log \dot{g}_\alpha$, and state gradients if A_t, b_t are differentiated.
Previous TT marginal	$a_{t-1}, \dot{a}_{t-1}, \hat{z}_{t-1}, \dot{\hat{z}}_{t-1}$.
Affine/preconditioner branch	Either fixed numerically, or derivatives $\dot{A}_t, \dot{b}_t$ and $\text{tr}(A_t^{-1} \dot{A}_t)$.
Stabilizing constant	Either branch derivative $\dot{c}_t$ at the frozen selected grid point, or a renamed scalar with c_t fixed numerically.
TT core construction	Fixed interpolation or least-squares equations and $\dot{G}_{t,k}$ from (4) or (4).
Normalizer	Same value path and derivative path, preferably the direct mass contraction (5).
Branch diagnostics	Fixed ranks, fixed interpolation points, fixed samples, fixed truncation decisions, fixed QR/SVD signs, and finite positive normalizers.

8 What Is Not Derived

This note does not derive the derivative of:

- changing TT-cross interpolation sets or maxvol/pivot choices;
- random enrichment samples or debug-sample resampling;
- rank adaptation, SVD truncation rank changes, or sign flips at repeated singular values;
- ESS-triggered reapproximation and sample doubling;
- nonunique minimizers in the stabilizing constant c_t ;
- the full MATLAB/Octave companion code as an HMC-ready backend.

Those operations can still be valuable for filtering and transport. They are not, as stated, a smooth same-scalar gradient path for ordinary HMC.

9 Minimal Verification Protocol

For each external parameter coordinate α_i , a future implementation should run:

1. value check: the scalar used by HMC is exactly $\sum_t \{\log \hat{z}_t - c_t\}$;
2. branch check: ranks, interpolation sets, sample arrays, selected constant point, QR/SVD sign convention, and basis/domain maps are unchanged under $\alpha_i \pm h$;
3. finite-difference parity:

$$\frac{\hat{\ell}_T^{\text{TT}}(\alpha + he_i) - \hat{\ell}_T^{\text{TT}}(\alpha - he_i)}{2h} \approx \tilde{\ell}_T^{\text{TT}}(\alpha);$$

4. failure check: if branch objects change, report branch-local derivative failure rather than tuning HMC on the reported gradient.

10 Conclusion

The fixed-branch Zhao–Cui squared-TT scalar has a clear analytical derivative: differentiate the input density, differentiate the fixed TT core equations, contract the differentiated cores through the same mass matrices used by the normalizer, and subtract the derivative of the stabilizing constant used in the declared scalar. This is enough for a future implementation plan. It is not enough to claim that the adaptive companion code already supplies an HMC gradient.

References

- [1] Zhao, B. and Cui, T. (2024). Tensor-Train Methods for Sequential State and Parameter Learning in State-Space Models. *Journal of Machine Learning Research*, 25(272):1–83.
- [2] Cui, T. and Dolgov, S. (2022). Deep Composition of Tensor-Trains Using Squared Inverse Rosenblatt Transports. *Foundations of Computational Mathematics*, 22:1863–1922.